

September 16, 2026 at 08:43

1. Introduction. The flower snark clauses are one of Knuth’s favorite benchmarks. Exercise 176 of this same section defines the cubic graph J_q on $4q$ vertices and its line graph $L(J_q)$, which needs four colors when q is odd; the clauses $fsnark(q)$ that ask for a 3-coloring of $L(J_q)$ are therefore unsatisfiable, and they are hard. Section 7.2.2.2 uses them again and again.

Exercise 282 asks for a proof that a SAT solver can check:

282. [M33] Construct a certificate of unsatisfiability for the clauses $fsnark(q)$ of exercise 176 when $q \geq 3$ is odd, using $O(q)$ clauses, all having length ≤ 4 .

A *certificate of unsatisfiability* is a sequence of clauses $(C_1, \dots, C_t)$ with $C_t = \epsilon$ such that

$$F \wedge C_1 \wedge \dots \wedge C_{i-1} \wedge \overline{C}_i \vdash_1 \epsilon, \quad 1 \leq i \leq t, \quad (119)$$

where $\vdash_1$ means “unit propagation reaches a contradiction” and $\overline{C}_i$ is the conjunction of the complements of C_i ’s literals. Checking one is cheap and needs no trust in the solver that produced it, which is the whole point.

The answer builds such a certificate by hand, in five stages. This program builds the clauses exactly as the answer prescribes, and checks every one of them with a unit-propagation checker of its own—no solver anywhere in that loop. Then it turns the tables: it asks the `sat` package of the directory above, a Go rendering of Knuth’s SAT13, to refute $fsnark(q)$ itself, and checks the certificate that Algorithm C produces as a byproduct (Theorem G). The `1` option of SAT13, which writes the learned clauses to a file, was ported to the package while this reading was being written; it is *SetProof* there.

I wrote this on 16 September 2026.

2. Every claim is a mode.

```

package main
import (
    "context"
    "flag"
    "fmt"
    "math/rand/v2"
    "os"
    "strconv"
    "strings"
    "time"
    "github.com/sjnam/sat"
)
⟨Types 15⟩
⟨Global variables 4⟩
⟨Functions 5⟩
func main() {
    ⟨Read the command line 3⟩
    if on("graph") {
        ⟨Check the graph against exercise 176 10⟩
    }
    if on("fsnark") {
        ⟨Generate the clauses and solve them 20⟩
    }
    if on("cert") {
        ⟨Build the answer's certificate and check it 45⟩
    }
    if on("remarks") {
        ⟨Check what the answer says in passing 47⟩
    }
    if on("algc") {
        ⟨Check the certificate that Algorithm C produces 49⟩
    }
    if on("waarden") {
        ⟨Refute waerden(3,10;97) and check its certificate 52⟩
    }
    if on("selftest") {
        ⟨Test the checker itself 54⟩
    }
    if mode == "dump" {
        fmt.Print(knuthFile())
    }
}

```

3. The default q is 9, which makes every mode instant. The interesting value is 99, the size Knuth uses in the text; `-mode algc -q 99` takes a few seconds.

⟨ Read the command line 3 ⟩ $\equiv$

```
flag.StringVar(&mode, "mode", "all",
    "graph, fsnark, cert, remarks, algc, waerden, selftest, all, or dump")
flag.IntVar(&q, "q", 9, "the order of the flower snark")
flag.Parse()
if q < 3 {
    fmt.Println("q must be at least 3")
    os.Exit(1)
}
```

This code is used in section 2.

4. ⟨ Global variables 4 ⟩ $\equiv$

```
var (
    mode string
    q int
)
```

This code is also defined in section 6.

This code is used in section 2.

5. ⟨ Functions 5 ⟩ $\equiv$

```
func on(m string) bool { return mode  $\equiv$  m  $\vee$  mode  $\equiv$  "all" }
```

This code is also defined in sections 7, 8, 9, 13, 16, 17, 19, 22, 23, 24, 25, 36, 37, 38, 40, 41, 42, 43, 51, 55, 60, 61.

This code is used in section 2.

6. The flower snark and its line graph. The vertices of J_q are t_j, u_j, v_j, w_j and its edges are $t_j - t_{j+1}, t_j - u_j, u_j - v_j, u_j - w_j, v_j - w_{j+1}, w_j - v_{j+1}$, subscripts modulo q . Answer 176(a) names the corresponding vertices of the line graph $a_j, b_j, c_j, d_j, e_j, f_j$ and lists its edges; those twelve per index are what this program uses, and the graph mode checks them against J_q itself.

⟨Global variables 4⟩ +≡

```
const (
    aVert = iota
    bVert
    cVert
    dVert
    eVert
    fVert
)
```

7. Vertex x_j of the line graph is number $6(j-1) + x$, with j reduced modulo q first, exactly as in Knuth's generator.

⟨Functions 5⟩ +≡

```
func vert(x, j int) int {
    j = (j - 1) % q + 1
    if j < 1 {
        j += q
    }
    return 6 * (j - 1) + x
}

func vertName(v int) string {
    return fmt.Sprintf("%c%d", 'a' + v % 6, v/6 + 1)
}
```

8. Here are the twelve edges of answer 176(a) that belong to index j , in the order Knuth's program creates them.

⟨Functions 5⟩ +≡

```
func edgesAt(j int) [][]int {
    k := j + 1
    return [][]int{
        {vert(aVert, j), vert(aVert, k)}, {vert(aVert, j), vert(bVert, j)},
        {vert(aVert, j), vert(bVert, k)}, {vert(bVert, j), vert(cVert, j)},
        {vert(bVert, j), vert(dVert, j)}, {vert(cVert, j), vert(dVert, j)},
        {vert(cVert, j), vert(eVert, j)}, {vert(dVert, j), vert(fVert, j)},
        {vert(eVert, j), vert(dVert, k)}, {vert(eVert, j), vert(fVert, k)},
        {vert(fVert, j), vert(cVert, k)}, {vert(fVert, j), vert(eVert, k)},
    }
}

func edges() [][]int {
    var es [][]int
    for j := 1; j ≤ q; j++ {
        es = append(es, edgesAt(j)...)
    }
    return es
}
```

9. The line graph of a cubic graph is 4-regular, and its $12q$ edges fall into $4q$ triangles—the two at each vertex of J_q and the ones coming from the edges. The answer names them $\{b_j, c_j, d_j\}$, $\{a_j, a_{j'}, b_{j'}\}$, $\{f_j, e_{j'}, c_{j'}\}$, $\{e_j, f_{j'}, d_{j'}\}$, where $j' = j + 1$; the program checks that these really are triangles and that they cover every edge exactly once.

⟨ Functions 5 ⟩ $\equiv$

```

func triangles() [[3]int {
  var ts [[3]int
  for j := 1; j ≤ q; j++ {
    k := j + 1
    ts = append(ts,
      [3]int{vert(bVert, j), vert(cVert, j), vert(dVert, j)},
      [3]int{vert(aVert, j), vert(aVert, k), vert(bVert, k)},
      [3]int{vert(fVert, j), vert(eVert, k), vert(cVert, k)},
      [3]int{vert(eVert, j), vert(fVert, k), vert(dVert, k)})
  }
  return ts
}

```

10. The line graph is built here from J_q 's own edges, independently of the list above, and the two are compared. Two edges of J_q are adjacent in $L(J_q)$ when they share an endpoint.

⟨ Check the graph against exercise 176 10 ⟩ $\equiv$

```

⟨ Build  $L(J_q)$  from  $J_q$  11 ⟩
mine := map[[2]int]bool{ }
for _, e := range edges() {
  mine[normalize(e)] = true
}
fmt.Printf("L(J_%d) : %d edges from J_%d, %d from answer_176(a), same: %v\n",
  q, len(theirs), q, len(mine), sameEdges(theirs, mine))
⟨ Check the degrees and the triangles 14 ⟩

```

This code is used in section 2.

11. ⟨ Build $L(J_q)$ from J_q 11 ⟩ $\equiv$

```

label := map[[2]int]int{ } // an edge of  $J_q$  and the vertex of  $L(J_q)$  it becomes
for j := 1; j ≤ q; j++ {
  k := j % q + 1
  ⟨ Name the six edges of  $J_q$  that belong to index j 12 ⟩
}
theirs := map[[2]int]bool{ }
for e1, v1 := range label {
  for e2, v2 := range label {
    if v1 < v2 ∧ (e1[0] ≡ e2[0] ∨ e1[0] ≡ e2[1] ∨
      e1[1] ≡ e2[0] ∨ e1[1] ≡ e2[1]) {
      theirs[[2]int{v1, v2}] = true
    }
  }
}

```

This code is used in section 10.

12. The four vertices of J_q at index j are numbered $4(j-1)+0, \dots, 3$ for t_j, u_j, v_j, w_j .

⟨Name the six edges of J_q that belong to index j 12⟩ $\equiv$

```

t, u, v, w := 4 * (j - 1), 4 * (j - 1) + 1, 4 * (j - 1) + 2, 4 * (j - 1) + 3
tk, vk, wk := 4 * (k - 1), 4 * (k - 1) + 2, 4 * (k - 1) + 3
label[normalize([2]int{t, tk})] = vert(aVert, j)
label[normalize([2]int{t, u})] = vert(bVert, j)
label[normalize([2]int{u, v})] = vert(cVert, j)
label[normalize([2]int{u, w})] = vert(dVert, j)
label[normalize([2]int{v, wk})] = vert(eVert, j)
label[normalize([2]int{w, vk})] = vert(fVert, j)

```

This code is used in section 11.

13. ⟨Functions 5⟩ $+\equiv$

```

func normalize(e [2]int) [2]int {
  if e[0] > e[1] {
    return [2]int{e[1], e[0]}
  }
  return e
}

func sameEdges(a, b map[[2]int]bool) bool {
  if len(a) != len(b) {
    return false
  }
  for e := range a {
    if !b[e] {
      return false
    }
  }
  return true
}

```

14. Every vertex should have degree four, and the $4q$ triangles should cover each of the $12q$ edges exactly once.

⟨ Check the degrees and the triangles 14 ⟩ $\equiv$

```

deg := make([],int,6 * q)
for _, e := range edges() {
    deg[e[0]]++
    deg[e[1]]++
}
four := true
for _, d := range deg {
    four = four ∧ d ≡ 4
}
cover := map[[2]int]int{ }
for _, t := range triangles() {
    for _, e := range [[2]int] { {t[0], t[1]}, {t[0], t[2]}, {t[1], t[2]} } {
        cover[normalize(e)]++
    }
}
once := len(cover) ≡ 12 * q
for _, n := range cover {
    once = once ∧ n ≡ 1
}
fmt.Printf("every vertex has degree 4: %v; the %d triangles cover " +
    "every edge exactly once: %v\n", four, len(triangles()), once)

```

This code is used in section 10.

15. The clauses. A clause is a list of literals, and a literal is $2v+1$ for $\bar{v}$ and $2v$ for v , where the variable $v.p$ (“vertex v has color p ”) is numbered $3v+p-1$.

⟨Types 15⟩ $\equiv$

```
type clause []int
```

This code is also defined in section 35.

This code is used in section 2.

16. ⟨Functions 5⟩ $+\equiv$

```
func lit( $v, p$  int,  $neg$  bool) int {
     $l := 2 * (3 * v + p - 1)$ 
    if  $neg$  {
         $l++$ 
    }
    return  $l$ 
}

func litName( $l$  int) string {
     $v, p := l \gg 1/3, l \gg 1 \% 3 + 1$ 
     $s := fmt.Sprintf("%s.%d", vertName(v), p)$ 
    if  $l \& 1 \equiv 1$  {
        return " $\sim$ " +  $s$ 
    }
    return  $s$ 
}
```

17. Knuth’s SAT-COLOR writes n clauses (15) saying that every vertex has a color, then mc clauses (16) saying that adjacent vertices differ, with the color as the outer loop; his change file prepends the three unit clauses $b_{1,1}$, $c_{1,2}$, $d_{1,3}$ that break the symmetry. The clauses come out in exactly that order here, because the order decides the variable numbering and hence every mem the solver spends.

⟨Functions 5⟩ $+\equiv$

```
func fsnarkClauses() []clause {
     $cls := []\text{clause}\{\{lit(vert(bVert, 1), 1, false)\},$ 
         $\{lit(vert(cVert, 1), 2, false)\}, \{lit(vert(dVert, 1), 3, false)\}\}$ 
    for  $v := 0; v < 6 * q; v++$  {
         $cls = append(cls, \text{clause}\{lit(v, 1, false), lit(v, 2, false),$ 
             $lit(v, 3, false)\})$ 
    }
    ⟨Append the clauses that keep neighbors apart 18⟩
    return  $cls$ 
}
```

18. The neighbors of a vertex are visited in the order of the SGB arc list, which is the reverse of the order in which the edges were created, and only the neighbor with the larger number is used. This is what makes the file identical to Knuth's, byte for byte.

⟨ Append the clauses that keep neighbors apart 18 ⟩ ≡

```

arcs := make([][]int, 6 * q)
for _, e := range edges() {
    arcs[e[0]] = append([]int{e[1]}, arcs[e[0]]...)
    arcs[e[1]] = append([]int{e[0]}, arcs[e[1]]...)
}
for p := 1; p ≤ 3; p++ {
    for v := 0; v < 6 * q; v++ {
        for _, u := range arcs[v] {
            if u > v {
                cls = append(cls, clause{lit(v, p, true), lit(u, p, true)})
            }
        }
    }
}

```

This code is used in section 17.

19. Knuth's file writes a unit clause with no leading blank, a clause of (15) with one blank before each literal, and a clause of (16) with none. The first line is a comment naming the generator. The `dump` mode prints it, so that it can be compared with `fsnark-99.sat` of Knuth's `SATexamples`; it agrees.

⟨ Functions 5 ⟩ +≡

```

func knuthFile() string {
    var b strings.Builder
    fmt.Fprintf(&b, "~_sat-color-snark1_%d\n", q)
    for _, c := range fsnarkClauses() {
        var names []string
        for _, l := range c {
            names = append(names, litName(l))
        }
        if len(c) == 3 { // a clause of (15): a blank before each literal
            b.WriteByte(' ')
            b.WriteString(strings.Join(names, " "))
        } else { // a unit clause, or a clause of (16)
            b.WriteString(strings.Join(names, ""))
        }
        b.WriteByte('\n')
    }
    return b.String()
}

```

20. Now the clauses can be solved. For odd q they are unsatisfiable; for even q they are not, which is exercise 176(b) and (c).

⟨ Generate the clauses and solve them 20 ⟩ $\equiv$

```

cls := fsnarkClauses()
fmt.Printf("fsnark(%d): %d variables, %d clauses\n", q, 18 * q, len(cls))
st, stats, _ := solve(cls, false)
fmt.Printf(" %v, %d+ %d mems, %d clauses learned\n", st, stats.IMems,
  stats.Mems, stats.Learned)
⟨ Solve a few small cases of both parities 21 ⟩

```

This code is used in section 2.

21. ⟨ Solve a few small cases of both parities 21 ⟩ $\equiv$

```

save := q
for q = 3; q ≤ 8; q ++ {
  st, _, _ := solve(fsnarkClauses(), false)
  want := sat.Unsat
  if q % 2 == 0 {
    want = sat.Sat
  }
  fmt.Printf(" %q= %d: %v (expected %v)\n", q, st, want)
}
q = save

```

This code is used in section 20.

22. The package numbers its variables in the order their names first appear, so adding the clauses in Knuth's order reproduces his numbering. With *withProof* the learned clauses are collected as they are produced.

⟨ Functions 5 ⟩ +≡

```

func solve(cls []clause, withProof bool)(sat.Status, sat.Stats, []clause) {
    s := sat.New()
    for _, c := range cls {
        var ls []sat.Lit
        for _, l := range c {
            x := s.Lookup(litName(l ⊙ 1))
            if l & 1 ≡ 1 {
                x = x.Not()
            }
            ls = append(ls, x)
        }
        s.AddClause(ls...)
    }
    var proof strings.Builder
    if withProof {
        s.SetProof(&proof)
    }
    st, err := s.Solve(context.Background())
    if err ≠ Λ {
        panic(err)
    }
    return st, s.Stats(), parseProof(proof.String(), litFromName)
}

```

23. The proof comes back as text in Knuth's format, so it has to be read back. A line that begins with two blanks is a clause that was shortened in place by on-the-fly subsumption; for our purposes it is just another learned clause. The names are turned back into the numbering used here, not into the solver's own, which follows the order in which names first appear.

⟨ Functions 5 ⟩ +≡

```

func parseProof(text string, litOf func(string) int) []clause {
    var out []clause
    for _, line := range strings.Split(text, "\n") {
        names := strings.Fields(line)
        if len(names) ≡ 0 {
            continue
        }
        var c clause
        for _, name := range names {
            c = append(c, litOf(name))
        }
        out = append(out, c)
    }
    return out
}

```

24. $\langle \text{Functions } 5 \rangle + \equiv$

```

func litFromName(name string) int {
    neg := false
    if name[0]  $\equiv$  '~' {
        neg, name = true, name[1:]
    }
    dot := strings.IndexByte(name, '.' )
    j, _ := strconv.Atoi(name[1:dot])
    p, _ := strconv.Atoi(name[dot + 1:])
    return lit(vert(int(name[0] - 'a'), j), p, neg)
}

```

25. The answer's certificate. The answer writes a_j for $a_{j,p}$, leaving the color p to be understood, and uses $j' = j + 1$ (modulo q) and $p' = p + 1$ (modulo 3). Its five stages are below, each in its own section, and *certificate* strings them together.

⟨ Functions 5 ⟩ +≡

```

func certificate(withBCD bool) []clause {
  var cert []clause
  add := func(ls ...int) { cert = append(cert, clause(ls)) }
  ⟨ Learn the exclusion clauses (17) 26 ⟩
  ⟨ Learn two clauses for every triangle 27 ⟩
  ⟨ Learn six clauses, and eighteen more, for every index 28 ⟩
  ⟨ Learn the clauses of the hint, working down from  $q$  31 ⟩
  ⟨ The endgame 33 ⟩
  return cert
}

```

26. “First learn the exclusion clauses (17).” They say that a vertex has at most one color; $f_{snark}(q)$ leaves them out, because a vertex with two colors is harmless. They are nevertheless implied, and unit propagation sees it: if v had two colors, the other two vertices of a triangle through v would both be forced into the third color. The answer remarks that the ones for b , c and d are never used, and *withBCD* omits them.

⟨ Learn the exclusion clauses (17) 26 ⟩ ≡

```

for v := 0; v < 6 * q; v++ {
  if ¬withBCD {
    switch v % 6 {
      case bVert, cVert, dVert:
        continue
    }
  }
  for p := 1; p ≤ 3; p++ {
    for r := p + 1; r ≤ 3; r++ {
      add(lit(v, p, true), lit(v, r, true))
    }
  }
}

```

This code is used in section 25.

27. “For every such triangle $\{u, v, w\}$, learn $(\bar{u}_{p'} \vee v_p \vee w_p)$ and then $(u_p \vee v_p \vee w_p)$.” The second clause says that every color appears in every triangle—true because the three vertices of a triangle have three distinct colors. The first is a stepping stone toward it.

⟨ Learn two clauses for every triangle 27 ⟩ ≡

```

for _, t := range triangles() {
  for p := 1; p ≤ 3; p++ {
    pp := p % 3 + 1
    add(lit(t[0], pp, true), lit(t[1], p, false), lit(t[2], p, false))
    add(lit(t[0], p, false), lit(t[1], p, false), lit(t[2], p, false))
  }
}

```

This code is used in section 25.

28. Now the certificate starts to look ahead along the ring. For each index it learns six clauses about a , e , f at j and j' , and for $j \geq 3$ eighteen more that tie the pattern at indices 1 and 2 to the pattern at j and j' . In the eighteen, u, v and u', v' each run through the three pairs (a, e) , (e, f) , (f, a) .

⟨ Learn six clauses, and eighteen more, for every index 28 ⟩ $\equiv$

```

pairs := [[2]int{{aVert, eVert}, {eVert, fVert}, {fVert, aVert}}
for j := 1; j ≤ q; j++ {
  k := j + 1
  for p := 1; p ≤ 3; p++ {
    ⟨ Learn the six clauses for index j and color p 29 ⟩
  }
  if j ≥ 3 {
    for p := 1; p ≤ 3; p++ {
      ⟨ Learn the eighteen clauses for index j and color p 30 ⟩
    }
  }
}

```

This code is used in section 25.

29. ⟨ Learn the six clauses for index j and color p 29 ⟩ $\equiv$

```

aj, ej, fj := vert(aVert, j), vert(eVert, j), vert(fVert, j)
ak, ek, fk := vert(aVert, k), vert(eVert, k), vert(fVert, k)
add(lit(aj, p, false), lit(fj, p, false), lit(ak, p, false), lit(ek, p, false))
add(lit(aj, p, false), lit(ej, p, false), lit(ak, p, false), lit(fk, p, false))
add(lit(ej, p, false), lit(fj, p, false), lit(ek, p, false), lit(fk, p, false))
add(lit(aj, p, true), lit(ej, p, true), lit(ek, p, true))
add(lit(aj, p, true), lit(fj, p, true), lit(fk, p, true))
add(lit(ej, p, true), lit(fj, p, true), lit(ak, p, true))

```

This code is used in section 28.

30. The odd case keeps two literals from the far index, the even case only one.

⟨ Learn the eighteen clauses for index j and color p 30 ⟩ $\equiv$

```

for _, uv := range pairs {
  for _, xy := range pairs {
    u1, v1 := vert(uv[0], 1), vert(uv[1], 1)
    u2, v2 := vert(uv[0], 2), vert(uv[1], 2)
    if j % 2 == 1 {
      add(lit(u1, p, true), lit(v1, p, true),
          lit(vert(xy[0], j), p, false), lit(vert(xy[1], j), p, false))
      add(lit(u2, p, true), lit(v2, p, true),
          lit(vert(xy[0], k), p, false), lit(vert(xy[1], k), p, false))
    } else {
      add(lit(u1, p, true), lit(v1, p, true), lit(vert(xy[0], j), p, true))
      add(lit(u2, p, true), lit(v2, p, true), lit(vert(xy[0], k), p, true))
    }
  }
}

```

This code is used in section 28.

31. “Then we’re ready to learn $(\bar{a}_j \vee \bar{e}_j)$, $(\bar{a}_j \vee \bar{f}_j)$, $(\bar{e}_j \vee \bar{f}_j)$ for $j \in \{1, 2\}$ and $(a_j \vee e_j \vee f_j \vee a_{j'})$, $(a_j \vee e_j \vee f_j)$ for $j \in \{1, q\}$.” The clauses of the hint say that a_j , e_j , f_j have three distinct colors and that all three colors occur among them. After the two starting indices, the same pair of clauses is learned for $j = q, q-1, \dots, 2$, which walks the conclusion all the way around the ring.

$\langle$ Learn the clauses of the hint, working down from q 31 $\rangle \equiv$

```

for  $p := 1; p \leq 3; p++$  {
  for  $-, j := \text{range } []\text{int } \{ 1, 2 \}$  {
     $\langle$  Learn that  $a_j, e_j, f_j$  differ 32  $\rangle$ 
  }
  for  $-, j := \text{range } []\text{int } \{ 1, q \}$  {
     $\text{add}(\text{lit}(\text{vert}(\mathbf{aVert}, j), p, \text{false}), \text{lit}(\text{vert}(\mathbf{eVert}, j), p, \text{false}),$ 
       $\text{lit}(\text{vert}(\mathbf{fVert}, j), p, \text{false}), \text{lit}(\text{vert}(\mathbf{aVert}, j+1), p, \text{false}))$ 
     $\text{add}(\text{lit}(\text{vert}(\mathbf{aVert}, j), p, \text{false}), \text{lit}(\text{vert}(\mathbf{eVert}, j), p, \text{false}),$ 
       $\text{lit}(\text{vert}(\mathbf{fVert}, j), p, \text{false}))$ 
  }
}
for  $j := q; j \geq 2; j--$  {
  for  $p := 1; p \leq 3; p++$  {
     $\langle$  Learn that  $a_j, e_j, f_j$  differ 32  $\rangle$ 
  }
  for  $p := 1; p \leq 3; p++$  {
     $\text{add}(\text{lit}(\text{vert}(\mathbf{aVert}, j-1), p, \text{false}), \text{lit}(\text{vert}(\mathbf{eVert}, j-1), p, \text{false}),$ 
       $\text{lit}(\text{vert}(\mathbf{fVert}, j-1), p, \text{false}), \text{lit}(\text{vert}(\mathbf{aVert}, j), p, \text{false}))$ 
     $\text{add}(\text{lit}(\text{vert}(\mathbf{aVert}, j-1), p, \text{false}), \text{lit}(\text{vert}(\mathbf{eVert}, j-1), p, \text{false}),$ 
       $\text{lit}(\text{vert}(\mathbf{fVert}, j-1), p, \text{false}))$ 
  }
}

```

This code is used in section 25.

32. $\langle$ Learn that a_j, e_j, f_j differ 32 $\rangle \equiv$

```

 $\text{add}(\text{lit}(\text{vert}(\mathbf{aVert}, j), p, \text{true}), \text{lit}(\text{vert}(\mathbf{eVert}, j), p, \text{true}))$ 
 $\text{add}(\text{lit}(\text{vert}(\mathbf{aVert}, j), p, \text{true}), \text{lit}(\text{vert}(\mathbf{fVert}, j), p, \text{true}))$ 
 $\text{add}(\text{lit}(\text{vert}(\mathbf{eVert}, j), p, \text{true}), \text{lit}(\text{vert}(\mathbf{fVert}, j), p, \text{true}))$ 

```

This code is used in section 31.

33. The endgame is where the parity of q finally bites. Answer 176(c) shows that the colors of $a_j e_j f_j$ are permuted by an odd permutation whenever j is even, so going once around a ring of odd length is impossible. The certificate says this by fixing the colors of a_1 and e_1 and walking around; the three clauses learned at index j depend on the parity of j , and the walk closes up on a contradiction.

⟨ The endgame 33 ⟩ $\equiv$

```

for  $p := 1; p \leq 3; p++$  {
  for  $-, pr := \text{range } [][2]\text{int}$   $\{\{p \% 3 + 1, (p + 1) \% 3 + 1\}, \{(p + 1) \% 3 + 1, p \% 3 + 1\}\}$  {
     $p1, p2 := pr[0], pr[1]$ 
     $base := []\text{int}\{\text{lit}(\text{vert}(\mathbf{aVert}, 1), p, \mathbf{true}), \text{lit}(\text{vert}(\mathbf{eVert}, 1), p1, \mathbf{true})\}$ 
    for  $j := 2; j \leq q; j++$  {
      ⟨ Learn three clauses for index  $j$  of the walk 34 ⟩
    }
     $add(base\dots)$ 
  }
   $add(\text{lit}(\text{vert}(\mathbf{aVert}, 1), p, \mathbf{true}))$ 
}

```

This code is used in section 25.

34. ⟨ Learn three clauses for index j of the walk 34 ⟩ $\equiv$

```

 $aj, ej := \text{vert}(\mathbf{aVert}, j), \text{vert}(\mathbf{eVert}, j)$ 
 $trio := [][2]\text{int}\{\{p, p2\}, \{p1, p\}, \{p2, p1\}\}$ 
if  $j \% 2 \equiv 1$  {
   $trio = [][2]\text{int}\{\{p, p1\}, \{p1, p2\}, \{p2, p\}\}$ 
}
for  $-, pq := \text{range } trio$  {
   $add(\text{append}(\text{append}(\mathbf{clause}\{\}, base\dots), \text{lit}(aj, pq[0], \mathbf{true}),$ 
     $\text{lit}(ej, pq[1], \mathbf{false}))\dots)$ 
}

```

This code is used in section 33.

35. Checking a certificate. The checker is a miniature solver that knows only unit propagation: clauses watch two literals each, and a clause whose watched literal becomes false either finds a new watch, forces its other watch, or reports a conflict. This is the whole of condition (119), and none of the package’s machinery is involved.

⟨Types 15⟩ +≡

```

type checker struct {
    nv int
    cls []clause
    watch [][]int // clauses watching a literal
    val []int8 // 1 if the literal is true
    reason []int // the clause that forced a literal, -1 if none
    trail []int
    dead bool // the clauses at hand are already contradictory
}

```

36. ⟨Functions 5⟩ +≡

```

func newChecker(nv int, cls []clause) *checker {
    c := &checker{nv: nv, watch: make([][]int, 2 * nv), val: make([]int8, 2 * nv),
        reason: make([]int, 2 * nv)}
    for i := range c.reason {
        c.reason[i] = -1
    }
    for _, cl := range cls {
        c.add(cl)
    }
    if c.propagate() ≥ 0 {
        c.dead = true
    }
    return c
}

```

37. A unit clause is asserted at once; longer clauses watch their first two literals.

⟨ Functions 5 ⟩ +≡

```

func (c *checker) add(cl clause) int {
    k := len(c.cls)
    c.cls = append(c.cls, cl)
    switch len(cl) {
    case 0:
        c.dead = true
    case 1:
        if c.assign(cl[0], k) {
            c.dead = true
        }
    default:
        c.watch[cl[0]] = append(c.watch[cl[0]], k)
        c.watch[cl[1]] = append(c.watch[cl[1]], k)
    }
    return k
}

func (c *checker) assign(l, why int) bool {
    if c.val[l] ≡ 1 {
        return false
    }
    if c.val[l ⊕ 1] ≡ 1 {
        return true // the clauses at hand contradict each other
    }
    c.val[l], c.reason[l] = 1, why
    c.trail = append(c.trail, l)
    return false
}

```

38. Propagation returns the number of a falsified clause, or -1 .

⟨ Functions 5 ⟩ +≡

```

func (c *checker) propagate() int {
    for i := 0; i < len(c.trail); i++ {
        l := c.trail[i]
        ws := c.watch[l ⊕ 1]
        for j := 0; j < len(ws); j++ {
            ⟨ Look at clause ws[j], whose watched literal just went false 39 ⟩
        }
    }
    return -1
}

```

39. $\langle \text{Look at clause } ws[j], \text{ whose watched literal just went false } 39 \rangle \equiv$

```

k := ws[j]
cl := c.cls[k]
if cl[0]  $\equiv l \oplus 1$  {
    cl[0], cl[1] = cl[1], cl[0]
}
if c.val[cl[0]]  $\equiv 1$  {
    continue
}
moved := false
for m := 2; m < len(cl); m++ {
    if c.val[cl[m]  $\oplus 1$ ]  $\neq 1$  {
        cl[1], cl[m] = cl[m], cl[1]
        c.watch[cl[1]] = append(c.watch[cl[1]], k)
        ws[j] = ws[len(ws) - 1]
        ws = ws[:len(ws) - 1]
        c.watch[l  $\oplus 1$ ] = ws
        j --
        moved = true
    }
    break
}
if moved {
    continue
}
if c.val[cl[0]  $\oplus 1$ ]  $\equiv 1 \vee c.assign(cl[0], k)$  {
    return k
}

```

This code is used in section 38.

40. $\langle \text{Functions } 5 \rangle + \equiv$

```

func (c *checker) undo(to int) {
    for i := len(c.trail) - 1; i  $\geq to$ ; i -- {
        c.val[c.trail[i]] = 0
        c.reason[c.trail[i]] = -1
    }
    c.trail = c.trail[:to]
}

```

41. Here is the test of (119) for one clause. Its literals are negated, the consequences are drawn, and a conflict must appear. What is left behind is the list of clauses that took part, which exercise 284 uses to tell which clauses of a certificate are really needed.

⟨ Functions 5 ⟩ +≡

```

func (c *checker) rup(cl clause)(bool, []int) {
    if c.dead {
        return true,  $\Lambda$ 
    }
    base := len(c.trail)
    conflict, forced := -1, -1
    for _, l := range cl {
        if c.val[l] == 1 { // its negation contradicts what we know
            forced = c.reason[l]
            break
        }
        if c.val[l  $\oplus$  1] != 1 {
            c.assign(l  $\oplus$  1, -1)
        }
    }
    var used []int
    if forced == -1 {
        conflict = c.propagate()
        if conflict  $\geq$  0 {
            used = c.participants(conflict)
        }
    } else if forced  $\geq$  0 {
        used = []int{forced}
    }
    c.undo(base)
    return forced != -1  $\vee$  conflict  $\geq$  0, used
}

```

42. To see which clauses really mattered, start from the falsified clause and walk back down the trail, taking the reason of every literal that the walk has touched. This is the marking of exercise 284, done with reasons so that clauses which propagated something irrelevant are not marked.

⟨ Functions 5 ⟩ +≡

```

func (c *checker) participants(conflict int) []int {
    seen := map[int]bool{ }
    used := []int{ conflict }
    for _, m := range c.cls[conflict] {
        seen[m ⊕ 1] = true
    }
    for i := len(c.trail) - 1; i ≥ 0; i -- {
        l := c.trail[i]
        if ¬seen[l] ∨ c.reason[l] < 0 {
            continue
        }
        used = append(used, c.reason[l])
        for _, m := range c.cls[c.reason[l]] {
            if m ≠ l {
                seen[m ⊕ 1] = true
            }
        }
    }
    return used
}

```

43. A whole certificate is checked in one forward pass. Afterwards the clauses that the final contradiction really depends on are collected backwards, which is how exercise 284 saves work; here it only reports how much of the certificate was redundant.

⟨ Functions 5 ⟩ +≡

```

func check(nv int, cls, cert []clause)(bad, used int, need []bool,
    dur time.Duration) {
    start := time.Now()
    c := newChecker(nv, cls)
    base := len(c.cls)
    deps := make([][]int, len(cert))
    for i, cl := range cert {
        ok, d := c.rup(cl)
        if ¬ok {
            bad++
        }
        deps[i] = d
        c.add(cl)
    }
    ⟨ Make sure the empty clause follows, and count what was needed 44 ⟩
    return bad, used, need[base:], time.Since(start)
}

```

44. $\langle$ Make sure the empty clause follows, and count what was needed 44 $\rangle \equiv$

```

ok, last := c.rup( $\Lambda$ )
if  $\neg ok$  {
    bad ++
}
need = make([]bool, len(c.cls))
for  $\_, k$  := range last {
    need[k] = true
}
for i := len(cert) - 1; i  $\geq$  0; i -- {
    if  $\neg need[base + i]$  {
        continue
    }
    used ++
    for  $\_, k$  := range deps[i] {
        need[k] = true
    }
}

```

This code is used in section 43.

45. What the answer claims. The certificate mode builds the clauses and checks them. The exercise asks for $O(q)$ clauses of length at most four, so both are reported; the count comes to $147q - 102$.

⟨ Build the answer's certificate and check it 45 ⟩ ≡

```

cls := fsnarkClauses()
cert := certificate(true)
long := 0
for _, c := range cert {
    if len(c) > long {
        long = len(c)
    }
}
bad, used, _, dur := check(18 * q, cls, cert)
fmt.Printf("certificate_for_q=%d: %d clauses, longest %d, failures %d, " +
    "needed %d, checked in %v\n", q, len(cert), long, bad, used,
    dur.Round(time.Millisecond))
⟨ Report how the count grows with q 46 ⟩

```

This code is used in section 2.

46. ⟨ Report how the count grows with q 46 ⟩ ≡

```

save := q
fmt.Printf(" clauses_for_q=3,5,...,15:")
for q = 3; q ≤ 15; q += 2 {
    fmt.Printf("%d", len(certificate(true)))
}
fmt.Printf("; that is 147q-102\n")
q = save

```

This code is used in section 45.

47. The answer adds three remarks, and each one is a claim to check: that the exclusion clauses for b , c and d are never used; that the certificate does not lean on the three symmetry-breaking unit clauses of $fsnark(q)$; and, implicitly, that q must be odd.

⟨ Check what the answer says in passing 47 ⟩ ≡

```

cls := fsnarkClauses()
bad, used, _, _ := check(18 * q, cls, certificate(false))
fmt.Printf("without the exclusion clauses for b, c, d: %d clauses, " +
    "failures %d, needed %d\n", len(certificate(false)), bad, used)
bad, _, _, _ = check(18 * q, cls[3:], certificate(true))
fmt.Printf("against fsnark(%d) without its three unit clauses: failures %d\n",
    q, bad)
⟨ Try an even order 48 ⟩

```

This code is used in section 2.

48. $\langle \text{Try an even order 48} \rangle \equiv$

```

save := q
q++
bad, -, _ = check(18 * q, fsnarkClauses(), certificate(true))
st, -, _ := solve(fsnarkClauses(), false)
fmt.Printf("at q=%d, which is even: %v, and the certificate has %d failures\n",
    q, st, bad)
q = save

```

This code is used in section 47.

49. Algorithm C's own certificate. Theorem G says that the clauses Algorithm C learns are themselves a certificate of unsatisfiability. The package now hands them over through *SetProof*, so the claim can be tested rather than believed; and once they are in hand, the two certificates can be compared.

⟨ Check the certificate that Algorithm C produces 49 ⟩ ≡

```

cls := fsnarkClauses()
st, stats, cert := solve(cls, true)
if st ≠ sat.Unsat {
    fmt.Println("expected UNSAT, got", st)
} else {
    ⟨ Report and check what Algorithm C learned 50 ⟩
}

```

This code is used in section 2.

50. ⟨ Report and check what Algorithm C learned 50 ⟩ ≡

```

sum, long := 0, 0
for _, c := range cert {
    sum += len(c)
    if len(c) > long {
        long = len(c)
    }
}
bad, used, _, dur := check(18 * q, cls, cert)
fmt.Printf("Algorithm C on fsnark(%d): %d+ %d mems, %d clauses learned\n",
    q, stats.IMems, stats.Mems, stats.Learned)
fmt.Printf("certificate: %d clauses, average length %.1f, longest %d\n",
    len(cert), float64(sum)/float64(len(cert)), long)
fmt.Printf("failures %d, needed %d (%.1f%%), checked in %v\n", bad, used,
    100 * float64(used)/float64(len(cert)), dur.Round(time.Millisecond))
fmt.Printf("the answer's certificate has %d clauses, none longer than 4\n",
    len(certificate(true)))

```

This code is used in section 49.

51. The other example in the text. Page 255, where certificates are introduced, gives numbers for two runs: Algorithm C learns “about 53,000” clauses when it refutes *waerden*(3, 10; 97) and “about 135,000” when it refutes *fsnark*(99), of which “fewer than 50,000” and “fewer than 47,000” are used again later; and the waerden certificate “was found in 272 megamems,” while checking it by straightforward unit propagation cost 2.2 gigamems. The flower snark half is measured by the **alg**c mode above. The other half needs the van der Waerden clauses, which are two lines to generate: no j equally spaced positions may all be 0, and no k equally spaced positions may all be 1.

⟨ Functions 5 ⟩ +≡

```

func waerdenClauses(j, k, n int) []clause {
  var cls []clause
  runs := func(length int, neg bool) {
    for d := 1; (length - 1) * d < n; d++ {
      for i := 1; i + (length - 1) * d ≤ n; i++ {
        var c clause
        for t := 0; t < length; t++ {
          l := 2 * (i + t * d - 1)
          if neg {
            l++
          }
          c = append(c, l)
        }
        cls = append(cls, c)
      }
    }
  }
  runs(j, false)
  runs(k, true)
  return cls
}

```

52. These clauses come out in the same order as Knuth’s own *waerden-3-10-97.sat*, so the solver spends the same mems on them and learns the same clauses.

⟨ Refute *waerden*(3,10;97) and check its certificate 52 ⟩ ≡

```

cls := waerdenClauses(3, 10, 97)
s, st, cert := solveRaw(97, cls, true)
if st ≠ sat.Unsat {
  fmt.Println("expected_␣UNSAT,␣got", st)
} else {
  ⟨ Report and check the waerden certificate 53 ⟩
}

```

This code is used in section 2.

53. $\langle$ Report and check the waerden certificate 53 $\rangle \equiv$

```

bad, used, -, dur := check(97, cls, cert)
stats := s.Stats()
fmt.Printf("waerden(3,10;97): %d clauses, %d+ %d mems, %d clauses learned\n",
    len(cls), stats.IMems, stats.Mems, stats.Learned)
fmt.Printf(" certificate of %d clauses\n", len(cert))
fmt.Printf(" failures %d, needed %d (%.1f%%), checked in %v\n", bad, used,
    100 * float64(used)/float64(len(cert)), dur.Round(time.Millisecond))

```

This code is used in section 52.

54. Testing the checker. A checker that never says no is worth nothing. The decisive test is this: if a formula has a satisfying assignment, then no clause that the assignment falsifies can be implied by it, so unit propagation must fail to derive a contradiction from it. Random satisfiable problems therefore give a supply of clauses that the checker has to reject, and it must reject every one.

A second test is the other way round: genuine certificates of the unsatisfiable problems must pass. Spoiling one is not a test, because a certificate carries slack—dropping a clause, or changing a literal, often leaves a sequence that is still valid, since the same conclusions can be reached along another path. What can be measured is how often the damage shows, and that is reported rather than insisted upon.

⟨Test the checker itself 54⟩ ≡

```
rng := rand.New(rand.NewPCG(282, 2026))
⟨Reject clauses that a satisfying assignment falsifies 56⟩
⟨Accept genuine certificates, and see how easily they break 58⟩
```

This code is used in section 2.

55. Ten variables and thirty random clauses of three literals are usually satisfiable; forty-five are usually not.

⟨Functions 5⟩ +≡

```
func randomProblem(rng *rand.Rand, nv, m int) []clause {
    var cls []clause
    for i := 0; i < m; i++ {
        var c clause
        for len(c) < 3 {
            if l := rng.IntN(2 * nv); ¬hasVar(c, l) {
                c = append(c, l)
            }
        }
        cls = append(cls, c)
    }
    return cls
}

func hasVar(c clause, l int) bool {
    for _, m := range c {
        if m >> 1 ≡ l >> 1 {
            return true
        }
    }
    return false
}
```

56. $\langle \text{Reject clauses that a satisfying assignment falsifies 56} \rangle \equiv$

```

const nv = 10
rejected, offered := 0, 0
for trial := 0; trial < 200; trial++ {
  cls := randomProblem(rng, nv, 30)
  s, st, _ := solveRaw(nv, cls, false)
  if st ≠ sat.Sat {
    continue
  }
  c := newChecker(nv, cls)
  for k := 0; k < 5; k++ {
     $\langle \text{Fabricate a clause the solution falsifies, and offer it 57} \rangle$ 
  }
}
fmt.Printf("clauses_refuted_by_known_solution: %d of %d rejected\n",
  rejected, offered)

```

This code is used in section 54.

57. Every literal of the fabricated clause is false in the solution, so the clause cannot follow from the formula by any means, let alone by propagation.

$\langle \text{Fabricate a clause the solution falsifies, and offer it 57} \rangle \equiv$

```

var bogus clause
for len(bogus) < 1 + rng.IntN(3) {
  v := rng.IntN(nv)
  l := 2 * v
  if s.Value(sat.Pos(v + 1)) {
    l++ // the solution makes  $x_{v+1}$  true, so  $\bar{x}_{v+1}$  is false
  }
  if ¬hasVar(bogus, l) {
    bogus = append(bogus, l)
  }
}
offered++
if ok, _ := c.rup(bogus); ¬ok {
  rejected++
}

```

This code is used in section 56.

58. $\langle$ Accept genuine certificates, and see how easily they break 58 $\rangle \equiv$

```

good, wrong, spoiled, noticed := 0, 0, 0, 0
for trial := 0; trial < 200; trial++ {
  cls := randomProblem(rng, nv, 45)
  _, st, cert := solveRaw(nv, cls, true)
  if st ≠ sat.Unsat ∨ len(cert) < 3 {
    continue
  }
  good++
  bad, _, need, _ := check(nv, cls, cert)
  if bad ≠ 0 {
    wrong++
  }
   $\langle$  Spoil this certificate and see whether it is noticed 59  $\rangle$ 
}
fmt.Printf("genuine_certificates: %d checked, %d rejected\n", good, wrong)
fmt.Printf("spoiled_certificates: %d of %d were noticed\n", noticed, spoiled)

```

This code is used in section 54.

59. The three kinds of damage are: dropping a clause that the backward pass found necessary, changing one of its literals, and reversing the order.

$\langle$ Spoil this certificate and see whether it is noticed 59 $\rangle \equiv$

```

victim := -1
for i, ok := range need {
  if ok {
    victim = i
    break
  }
}
if victim < 0 {
  continue
}
dropped := append(append([], clause{ }, cert[:victim]...), cert[victim + 1:]...)
changed := append([], clause{ }, cert...)
changed[victim] = alter(rng, changed[victim], nv)
reversed := make([], clause, len(cert))
for i, cl := range cert {
  reversed[len(cert) - 1 - i] = cl
}
for _, mess := range [][]clause { dropped, changed, reversed } {
  spoiled++
  if bad, _, -, _ := check(nv, cls, mess); bad ≠ 0 {
    noticed++
  }
}

```

This code is used in section 58.

60. $\langle \text{Functions 5} \rangle + \equiv$

```

func alter(rng *rand.Rand, c clause, nv int) clause {
    out := append(clause{}, c...)
    for {
        if l := rng.IntN(2 * nv);  $\neg$ hasVar(out, l) {
            out[rng.IntN(len(out))] = l
            return out
        }
    }
}

```

61. The random problems have nothing to do with flower snarks, so they get their own solver call, with plain variable names; the solver itself comes back too, so that a solution can be read off it.

$\langle \text{Functions 5} \rangle + \equiv$

```

func solveRaw(nv int, cls []clause, withProof bool)(*sat.Solver, sat.Status,
    []clause) { // the solver carries the statistics
    s := sat.New()
    for i := 0; i < nv; i++ {
        s.Lookup(fmt.Sprintf("x%d", i + 1))
    }
    for _, c := range cls {
        var ls []sat.Lit
        for _, l := range c {
            ls = append(ls, sat.Pos(l >> 1 + 1) + sat.Lit(l & 1))
        }
        s.AddClause(ls...)
    }
    var proof strings.Builder
    if withProof {
        s.SetProof(&proof)
    }
    st, err := s.Solve(context.Background())
    if err ≠  $\Lambda$  {
        panic(err)
    }
    return s, st, parseProof(proof.String()), func(name string) int {
        neg := 0
        if name[0] ≡ '~' {
            neg, name = 1, name[1:]
        }
        v, _ := strconv.Atoi(name[1:])
        return 2 * (v - 1) + neg
    })
}

```

62. Index.

- a*: [13](#).
add: [25](#), [26](#), [27](#), [29](#), [30](#), [31](#), [32](#), [33](#), [34](#), [36](#), [37](#), [43](#).
AddClause: [22](#), [61](#).
aj: [29](#), [34](#).
ak: [29](#).
alter: [59](#), [60](#).
append: [8](#), [9](#), [17](#), [18](#), [19](#), [22](#), [23](#), [25](#), [34](#), [37](#), [39](#), [42](#),
[51](#), [55](#), [57](#), [59](#), [60](#), [61](#).
arcs: [18](#).
assign: [37](#), [39](#), [41](#).
Atoi: [24](#), [61](#).
aVert: [6](#), [8](#), [9](#), [12](#), [28](#), [29](#), [31](#), [32](#), [33](#), [34](#).
b: [13](#), [19](#).
Background: [22](#), [61](#).
bad: [43](#), [44](#), [45](#), [47](#), [48](#), [50](#), [53](#), [58](#), [59](#).
base: [33](#), [34](#), [41](#), [43](#), [44](#).
bogus: [57](#).
bool: [5](#), [10](#), [11](#), [13](#), [16](#), [22](#), [25](#), [35](#), [37](#), [41](#), [42](#), [43](#), [44](#),
[51](#), [55](#), [61](#).
Builder: [19](#), [22](#), [61](#).
bVert: [6](#), [8](#), [9](#), [12](#), [17](#), [26](#).
c: [19](#), [22](#), [23](#), [36](#), [37](#), [38](#), [39](#), [40](#), [41](#), [42](#), [43](#), [44](#), [45](#),
[50](#), [51](#), [55](#), [56](#), [57](#), [60](#), [61](#).
cert: [25](#), [43](#), [44](#), [45](#), [49](#), [50](#), [52](#), [53](#), [58](#), [59](#).
certificate: [25](#), [45](#), [46](#), [47](#), [48](#), [50](#).
changed: [59](#).
check: [43](#), [45](#), [47](#), [48](#), [50](#), [53](#), [58](#), [59](#).
checker: [35](#), [36](#), [37](#), [38](#), [40](#), [41](#), [42](#).
cl: [36](#), [37](#), [39](#), [41](#), [43](#), [59](#).
clause: [15](#), [17](#), [18](#), [22](#), [23](#), [25](#), [34](#), [35](#), [36](#), [37](#), [41](#),
[43](#), [51](#), [55](#), [57](#), [59](#), [60](#), [61](#).
cls: [17](#), [18](#), [20](#), [22](#), [35](#), [36](#), [37](#), [39](#), [42](#), [43](#), [44](#), [45](#), [47](#),
[49](#), [50](#), [51](#), [52](#), [53](#), [55](#), [56](#), [58](#), [59](#), [61](#).
conflict: [41](#), [42](#).
context: [22](#), [61](#).
cover: [14](#).
cVert: [6](#), [8](#), [9](#), [12](#), [17](#), [26](#).
d: [14](#), [43](#), [51](#).
dead: [35](#), [36](#), [37](#), [41](#).
deg: [14](#).
deps: [43](#), [44](#).
dot: [24](#).
dropped: [59](#).
dur: [43](#), [45](#), [50](#), [53](#).
Duration: [43](#).
dVert: [6](#), [8](#), [9](#), [12](#), [17](#), [26](#).
e: [10](#), [13](#), [14](#), [18](#).
e1: [11](#).
e2: [11](#).
edges: [8](#), [10](#), [14](#), [18](#).
edgesAt: [8](#).
ej: [29](#), [34](#).
ek: [29](#).
err: [22](#), [61](#).
es: [8](#).
eVert: [6](#), [8](#), [9](#), [12](#), [28](#), [29](#), [31](#), [32](#), [33](#), [34](#).
Exit: [3](#).
Fields: [23](#).
fj: [29](#).
fk: [29](#).
flag: [3](#).
float64: [50](#), [53](#).
fmt: [2](#), [3](#), [7](#), [10](#), [14](#), [16](#), [19](#), [20](#), [21](#), [45](#), [46](#), [47](#), [48](#),
[49](#), [50](#), [52](#), [53](#), [56](#), [58](#), [61](#).
forced: [41](#).
four: [14](#).
Fprintf: [19](#).
fsnarkClauses: [17](#), [19](#), [20](#), [21](#), [45](#), [47](#), [48](#), [49](#).
fVert: [6](#), [8](#), [9](#), [12](#), [28](#), [29](#), [31](#), [32](#).
good: [58](#).
hasVar: [55](#), [57](#), [60](#).
i: [36](#), [38](#), [40](#), [42](#), [43](#), [44](#), [51](#), [55](#), [59](#), [61](#).
IMems: [20](#), [50](#), [53](#).
IndexByte: [24](#).
int: [4](#), [7](#), [8](#), [9](#), [10](#), [11](#), [12](#), [13](#), [14](#), [15](#), [16](#), [18](#), [23](#), [24](#),
[25](#), [28](#), [31](#), [33](#), [34](#), [35](#), [36](#), [37](#), [38](#), [40](#), [41](#), [42](#), [43](#),
[51](#), [55](#), [60](#), [61](#).
int8: [35](#), [36](#).
IntN: [55](#), [57](#), [60](#).
IntVar: [3](#).
j: [7](#), [8](#), [9](#), [11](#), [12](#), [24](#), [28](#), [29](#), [30](#), [31](#), [32](#), [33](#), [34](#), [38](#),
[39](#), [51](#).
Join: [19](#).
k: [8](#), [9](#), [11](#), [12](#), [28](#), [29](#), [30](#), [37](#), [39](#), [44](#), [51](#), [56](#).
knuthFile: [2](#), [19](#).
l: [16](#), [19](#), [22](#), [37](#), [38](#), [39](#), [41](#), [42](#), [51](#), [55](#), [57](#), [60](#), [61](#).
label: [11](#), [12](#).
last: [44](#).
Learned: [20](#), [50](#), [53](#).
len: [10](#), [13](#), [14](#), [19](#), [20](#), [23](#), [37](#), [38](#), [39](#), [40](#), [41](#), [42](#), [43](#),
[44](#), [45](#), [46](#), [47](#), [50](#), [53](#), [55](#), [57](#), [58](#), [59](#), [60](#).
length: [51](#).
line: [23](#).
lit: [16](#), [17](#), [18](#), [24](#), [26](#), [27](#), [29](#), [30](#), [31](#), [32](#), [33](#), [34](#).

Lit: 22, 61.
litFromName: 22, 24.
litName: 16, 19, 22.
litOf: 23.
long: 45, 50.
Lookup: 22, 61.
ls: 22, 25, 61.
m: 5, 39, 42, 55.
main: 2.
make: 14, 18, 36, 43, 44, 59.
Mems: 20, 50, 53.
mess: 59.
Millisecond: 45, 50, 53.
mine: 10.
mode: 2, 3, 4, 5.
moved: 39.
n: 14, 51.
name: 23, 24, 61.
names: 19, 23.
need: 43, 44, 58, 59.
neg: 16, 24, 51, 61.
New: 22, 54, 61.
newChecker: 36, 43, 56.
NewPCG: 54.
normalize: 10, 12, 13, 14.
Not: 22.
noticed: 58, 59.
Now: 43.
nv: 35, 36, 43, 55, 56, 57, 58, 59, 60, 61.
offered: 56, 57.
ok: 43, 44, 57, 59.
on: 2, 5.
once: 14.
os: 3.
out: 23, 60.
p: 16, 18, 24, 26, 27, 28, 29, 30, 31, 32, 33, 34.
p1: 33, 34.
p2: 33, 34.
pairs: 28, 30.
panic: 22, 61.
Parse: 3.
parseProof: 22, 23, 61.
participants: 41, 42.
Pos: 57, 61.
pp: 27.
pq: 34.
pr: 33.
Print: 2.

Printf: 10, 14, 20, 21, 45, 46, 47, 48, 50, 53, 56, 58.
Println: 3, 49, 52.
proof: 22, 61.
propagate: 36, 38, 41.
q: 3, 4, 7, 8, 9, 10, 11, 14, 17, 18, 19, 20, 21, 26, 28, 31, 33, 45, 46, 47, 48, 50.
r: 26.
Rand: 55, 60.
rand: 54, 55, 60.
randomProblem: 55, 56, 58.
reason: 35, 36, 37, 40, 41, 42.
rejected: 56, 57.
reversed: 59.
rng: 54, 55, 56, 57, 58, 59, 60.
Round: 45, 50, 53.
runs: 51.
rup: 41, 43, 44, 57.
s: 16, 22, 52, 53, 56, 57, 61.
sameEdges: 10, 13.
Sat: 21, 56.
sat: 21, 22, 49, 52, 56, 57, 58, 61.
save: 21, 46, 48.
seen: 42.
SetProof: 1, 22, 49, 61.
Since: 43.
Solve: 22, 61.
solve: 20, 21, 22, 48, 49.
Solver: 61.
solveRaw: 52, 56, 58, 61.
Split: 23.
spoiled: 58, 59.
Sprintf: 7, 16, 61.
st: 20, 21, 22, 48, 49, 52, 56, 58, 61.
start: 43.
Stats: 22, 53.
stats: 20, 49, 50, 53.
Status: 22, 61.
strconv: 24, 61.
String: 19, 22, 61.
string: 4, 5, 7, 16, 19, 23, 24, 61.
strings: 19, 22, 23, 24, 61.
StringVar: 3.
sum: 50.
t: 12, 14, 27, 51.
text: 23.
theirs: 10, 11.
time: 43, 45, 50, 53.
tk: 12.

to: 40.
trail: 35, 37, 38, 40, 41, 42.
trial: 56, 58.
triangles: 9, 14, 27.
trio: 34.
ts: 9.
u: 12, 18.
u1: 30.
u2: 30.
undo: 40, 41.
Unsat: 21, 49, 52, 58.
used: 41, 42, 43, 44, 45, 47, 50, 53.
uv: 30.
v: 7, 12, 16, 17, 18, 26, 57, 61.
v1: 11, 30.
v2: 11, 30.
val: 35, 36, 37, 39, 40, 41.
Value: 57.
vert: 7, 8, 9, 12, 17, 24, 29, 30, 31, 32, 33, 34.
vertName: 7, 16.
victim: 59.
vk: 12.
w: 12.
waardenClauses: 51, 52.
want: 21.
watch: 35, 36, 37, 38, 39.
why: 37.
withBCD: 25, 26.
withProof: 22, 61.
wk: 12.
WriteByte: 19.
WriteString: 19.
wrong: 58.
ws: 38, 39.
x: 7, 22.
xy: 30.

- ⟨ Accept genuine certificates, and see how easily they break 58 ⟩ Used in section 54
- ⟨ Append the clauses that keep neighbors apart 18 ⟩ Used in section 17
- ⟨ Build $L(J_q)$ from J_q 11 ⟩ Used in section 10
- ⟨ Build the answer's certificate and check it 45 ⟩ Used in section 2
- ⟨ Check the certificate that Algorithm C produces 49 ⟩ Used in section 2
- ⟨ Check the degrees and the triangles 14 ⟩ Used in section 10
- ⟨ Check the graph against exercise 176 10 ⟩ Used in section 2
- ⟨ Check what the answer says in passing 47 ⟩ Used in section 2
- ⟨ Fabricate a clause the solution falsifies, and offer it 57 ⟩ Used in section 56
- ⟨ Functions 5 ⟩ Used in section 2
- ⟨ Generate the clauses and solve them 20 ⟩ Used in section 2
- ⟨ Global variables 4 ⟩ Used in section 2
- ⟨ Learn six clauses, and eighteen more, for every index 28 ⟩ Used in section 25
- ⟨ Learn that a_j , e_j , f_j differ 32 ⟩ Used in section 31
- ⟨ Learn the clauses of the hint, working down from q 31 ⟩ Used in section 25
- ⟨ Learn the eighteen clauses for index j and color p 30 ⟩ Used in section 28
- ⟨ Learn the exclusion clauses (17) 26 ⟩ Used in section 25
- ⟨ Learn the six clauses for index j and color p 29 ⟩ Used in section 28
- ⟨ Learn three clauses for index j of the walk 34 ⟩ Used in section 33
- ⟨ Learn two clauses for every triangle 27 ⟩ Used in section 25
- ⟨ Look at clause $ws[j]$, whose watched literal just went false 39 ⟩ Used in section 38
- ⟨ Make sure the empty clause follows, and count what was needed 44 ⟩ Used in section 43
- ⟨ Name the six edges of J_q that belong to index j 12 ⟩ Used in section 11
- ⟨ Read the command line 3 ⟩ Used in section 2
- ⟨ Refute waerden(3,10;97) and check its certificate 52 ⟩ Used in section 2
- ⟨ Reject clauses that a satisfying assignment falsifies 56 ⟩ Used in section 54
- ⟨ Report and check the waerden certificate 53 ⟩ Used in section 52
- ⟨ Report and check what Algorithm C learned 50 ⟩ Used in section 49
- ⟨ Report how the count grows with q 46 ⟩ Used in section 45
- ⟨ Solve a few small cases of both parities 21 ⟩ Used in section 20
- ⟨ Spoil this certificate and see whether it is noticed 59 ⟩ Used in section 58
- ⟨ Test the checker itself 54 ⟩ Used in section 2
- ⟨ The endgame 33 ⟩ Used in section 25
- ⟨ Try an even order 48 ⟩ Used in section 47
- ⟨ Types 15 ⟩ Used in section 2

A certificate for the flower snark

	Section	Page
Introduction	1	1
The flower snark and its line graph	6	4
The clauses	15	8
The answer's certificate	25	13
Checking a certificate	35	17
What the answer claims	45	23
Algorithm C's own certificate	49	25
The other example in the text	51	26
Testing the checker	54	28
Index	62	32